CMPUT 331, Assignment 6: Character n-grams

Before beginning work on this assignment, carefully read the Assignment Submission Specifications posted on Canvas.

Introduction

In this assignment, you will revisit the substitution cipher. In previous assignments, you saw cryptanalysis programs based on character repetition patterns within words (the textbook solver), and character frequencies (frequency analysis). The former tried to recognize patterns in the ciphertext characters, while the latter leveraged statistical information about English. Here, you will implement an approach to combine character sequence information with language statistics. This approach is based on sequences of characters, called *character n-grams*.

A character n-gram is a string consisting of n characters (n is simply a variable with a positive integer value) which appear in sequence in some string. For the purposes of this assignment, the set of all characters will be the 26 uppercase letters of the English alphabet and the 'space' character, giving a total of 27 characters. You need not handle punctuation, digits, or any other symbols other than uppercase letters and the space character.

For example the string "AN EXAMPLE" contains ten 1-grams (or "unigrams"), nine 2-grams ("bigrams"), eight 3-grams ("trigrams"), and so on, as shown in the table below (the '␣' symbol is used to make space characters easier to see):

n	n-grams in "AN EXAMPLE'
1	"A", "N", "␣", "E", ..., "E"
2	"AN", "N␣", "␣E", "EX", ..., "LE"
3	"AN␣", "'N E', "␣EX", "EXA", ..., "PLE"
4	"AN E", "N EX", "␣EXA", "EXAM", ..., "MPLE"
5	"AN EX", "N EXA", "␣EXAM", "EXAMP", ..., "AMPLE"

Just like individual characters, character n-grams have different relative frequencies in English. For example "TH" and "ING" are relatively common, while "TN" and "YYY" are relatively rare. In this assignment, you will implement a substitution cipher solver based on this linguistic principle. English n-gram frequencies are to be computed based on the included text file "wells.txt". The file "test.py" is provided to help you verify the functionality of your code.

Runtime Note: The general submission guidelines specify that each problem should run in under one second unless otherwise stated. For Assignment 6, the following runtime limits apply:

- Problem 1: Under 1 second.
- Problem 2: Under 1 second.
- Problem 3: Up to 10 seconds.
- Problem 4: Up to 60 seconds.

Please allow for these runtime limits when testing your solutions on lab machines. You will produce six (6) files for this assignment:

- python solutions to problems 1, 2, 3 and 4 (**a6p1.py**, **a6p2.py**, **a6p3.py**, **a6p4.py**)
- the decipherment produced in problem 4 (**text_plain.txt**)
- a README file (**README.txt**)

Submit your files in a zip file named **331-as-6.zip**.

Problem 1

Modify the module called `a6p1.py` to complete the function called `ngramsFreqs(text, n)`, which takes as input a string (the text content to analyze) and the n-gram integer variable n. It should return a dictionary in which each key is a character n-gram, represented as a single string, and the value of a key is the relative frequency (as a float) of that n-gram in the given text (the number of times that n-gram occurs, divided by the total number of character n-grams in the given text). Spaces should be treated as characters.

For example, `ngramsFreqs("ABABA", 2)` returns {`'AB': 0.5, 'BA': 0.5`}.

Problem 2

For the purpose of this assignment, a *mapping* will be a Python dictionary which maps our set of 27 characters to itself bijectively (so the mapping is exactly one-to-one, with 27 unique string keys and 27 unique string values), with the constraint that 'space' is always mapped to itself. For this problem these 27 keys and strings will simply be the 26 letters of the alphabet and the space character. Given a mapping, a ciphertext, and an n-gram frequency dictionary, the **n-gram score** of the key is computed as follows:

1. Attempt to decipher the ciphertext into a decipherment using the mapping, by mapping each ciphertext character to the value it has in that key.
2. For a given n-gram g, let $c(g)$ be the number of times it occurs in the decipherment, and let $f(g)$ be its relative frequency in the given frequency dictionary.
   (Note that: (1) $c(g)$ is an integer; (2) $f(g)$ is a float between 0 and 1; (3) $f(g) = 0$ if g is not in the frequency dictionary.)
3. Letting G be the set of all n-grams which occur in the decipherment, the score of the provided mapping is: $\sum_{g \in G} c(g) \cdot f(g)$

Example: Given the deciphered text "EEFFFEEF" and the frequency dictionary
EE: 0.5 EF: 0.2 FE: 0.3 where n = 2.
Broken down there are the following bigrams: EE, EF, FF, FF, FE, EE, EF
Following the n-gram score formula above
(2 * 0.5) + (2 * 0.2) + (2* 0) + (1 * 0.3) = 1.7 Therefore the score would be 1.7.

Modify the module *a6p2.py* to complete the function `keyScore( mapping, ciphertext, frequencies, n )`, which returns the n-gram score, as a floating-point number, computed given that mapping, ciphertext (given as a string), an n-gram frequency dictionary (such as is returned by your `ngramsFreqs` function), and the n-gram integer parameter n.

Problem 3

Given a mapping m, we can create a new mapping, call it m', by choosing two keys in the dictionary that represents m, and exchanging their values. For example, if m maps 'A' to 'X' and 'B' to 'Y', one such "swap" would have m' be identical to m, except that m' maps 'A' to 'Y' and 'B' to 'X'. Since a mapping has 26 keys which are not fixed (recall that any mapping must map space to itself), the total number of swaps is equal to 325 (there are 25 characters to swap 'A' with, 24 for 'B', and so on).

For a mapping m, let S_m be the set of all mappings which can be created by swapping two values in m in this way. We will call these mappings *the successors of* m. Each successor differs from the original mapping by only two letters. Some successor of m might have a higher score than m itself – that is, in terms of n-gram frequencies, it might be a better mapping.

Modify the module called *a6p3.py* to complete the function called `bestSuccessor( mapping, ciphertext, frequencies, n )`, which computes the n-gram score of each possible successor of the input mapping, given a string ciphertext, an n-gram frequency dictionary, and the value of n. If any such successor has a higher score than the given mapping, the function should return the successor with the highest such score. Otherwise, it should return the original mapping. A successor with a score equal to the original mapping's score is not considered an improvement. To maintain consistency when dealing with ties, you should use the provided `breakKeyScoreTie` function to obtain the better successor whenever two mappings have the same key score. Use `breakKeyScoreTie` only when two successors have the same score. If a successor has the same score as the original mapping, keep the original mapping.

Problem 4

Now we are ready to crack some ciphers! Use the module called *a6p4.py* to complete the function called `breakSub( cipherFile, textFile)`. Initially, this function should read the ciphertext from the input `cipherFile`, which is the path to the ciphertext file. After that, the function should create a mapping based on frequency analysis (so, the ith most frequent cipher character is mapped to the ith most frequent English character; the English frequency order is provided as a constant in the file). It should then repeatedly apply your `bestSuccessor` function, with the ciphertext being read in, a bigram frequency dictionary derived from the input textFile (the path to the given text file), halting when that function simply returns the same mapping it was given (that is, when a better mapping cannot be found through any swap). Once this happens, the function should generate the decipherment using that mapping and finally save the decipherment as a new text file.

This strategy of repeatedly replacing the current solution with the best of a set of successor solutions until no further improvement is possible is a strategy called *hill climbing*.

For submission, the `breakSub` function should take "text_cipher.txt" as the input cipherFile, "wells.txt" as the input textFile, and save the decipherment as an output text file named "text_plain.txt" for the submission (this location can be hard-coded).

Please keep solutions to problem 4 within a runtime of 1 minute when deciphering a file. Please review the plaintext for correctness before submitting. The program will be evaluated using bigrams, so ensure that your program produces the correct output.

Hint: you may experiment with different ways to define your initial mapping, using frequency as a starting point. One approach is to try both the actual English frequency and the one from the provided textfile.

How to Test

For Problem 1, Problem 2, and Problem 3, we have included a comprehensive set of test cases in the **test.py** file. We encourage you to utilize these test cases extensively to thoroughly validate your implementations before submitting your assignment. These test cases are designed to help you ensure the correctness and robustness of your solutions.

```
$ python test.py
Problem 1: Tests passed :)
Problem 2: Tests passed :)
Problem 3: Tests passed :)
```

For Problem 4, we have provided the test cipher file, "text_finnegan_cipher.txt". To validate your implementation, use this test cipher file as the input (cipherFile) and compare your resulting deciphered text in "text_plain.txt" with the provided reference file "text_finnegan_plain.txt". Following your testing, do not forget to execute your `breakSub` function on the main cipher file "text_cipher.txt" to generate the corresponding "text_plain.txt" for your final submission. You may find it useful to change the address of "text_plain.txt" in the code while testing.